

LLMs CAN'T PLAY HANGMAN: ON THE NECESSITY OF A PRIVATE WORKING MEMORY FOR LANGUAGE AGENTS

Davide Baldelli

Chandar Research Lab
LAMA-WeST Lab
Polytechnique Montréal
Mila – Quebec AI Institute
davide.baldelli@mila.quebec

Ali Parviz

University of California, San Diego
Mila – Quebec AI Institute
ali.parviz@mila.quebec

Amal Zouaq

LAMA-WeST Lab
Polytechnique Montréal
Mila – Quebec AI Institute
amal.zouaq@mila.quebec

Sarath Chandar

Chandar Research Lab
Polytechnique Montréal
Mila – Quebec AI Institute
sarath.chandar@mila.quebec

ABSTRACT

As LLMs move from text completion toward autonomous agents, they remain constrained by the standard chat interface, which lacks private working memory. This raises a fundamental question: can agents reliably perform interactive tasks that depend on hidden state? We define Private State Interactive Tasks (PSITs), which require agents to generate and maintain hidden information while producing public responses consistent with a fixed hidden state. We show theoretically that any agent restricted to the public conversation history cannot both keep the secret unresolved in the transcript and respond consistently with a fixed hidden state in PSITs, yielding an architectural impossibility theorem. To empirically validate this limitation, we introduce a self-consistency testing protocol that evaluates whether agents can maintain a hidden secret across forked dialogue branches. Standard chat-based LLMs and retrieval-based memory baselines fail this test regardless of scale, demonstrating that semantic retrieval does not enable true state maintenance. To address this, we propose a novel architecture incorporating an explicit private working memory; we demonstrate that this mechanism restores consistency with a fixed hidden state, establishing private state as a necessary component for PSIT-capable language agents. Our code is available at github.com/chandar-lab/Hangman.

1 INTRODUCTION

Large language models (LLMs) have demonstrated strong performance across a variety of natural language processing tasks and are increasingly deployed as the backbone of conversational agents (Brown et al., 2020; Achiam et al., 2023). Beyond text generation, recent research has extended LLMs into interactive and agentic settings, enabling tool use, planning, and long-horizon reasoning (Yao et al., 2023; Wang et al., 2024; Rozanov & Rei, 2024). These systems typically operate under a standard chat interface, where the model conditions only on the public dialogue history to produce its next response. While this paradigm has proven effective for tasks that can be solved with context-limited reasoning or external retrieval (Lewis et al., 2020; Borgeaud et al., 2022), it implicitly assumes that all relevant state can be represented in the shared conversation, and that the agent has no need for private information inaccessible to the user. However, as LLMs are increasingly used as interactive agents, many tasks require them to maintain information privately across turns, something the standard chat paradigm does not support. This reveals a critical gap in the current literature.

Despite progress in memory and reasoning for LLM-based agents, current systems lack mechanisms for maintaining private working memory. While reasoning models can generate hidden intermediate states, standard chat interfaces do not persist them: internal reasoning tokens are discarded between turns (OpenAI, 2025b;a; Hugging Face, 2025). As a result, any privately generated state is immediately lost. Consider, for instance, the game of Hangman, in which a host privately selects a secret word and must reveal the positions of correct letters as a player makes guesses. When playing Hangman as the host, a model may internally select a secret word, but this choice is forgotten by the next turn, forcing the agent to hallucinate a new state. We document this failure across major commercial interfaces - ChatGPT (powered by GPT-5.1), Gemini (powered by Gemini 3), and Claude (powered by Sonnet 4.5) - in Appendix D.

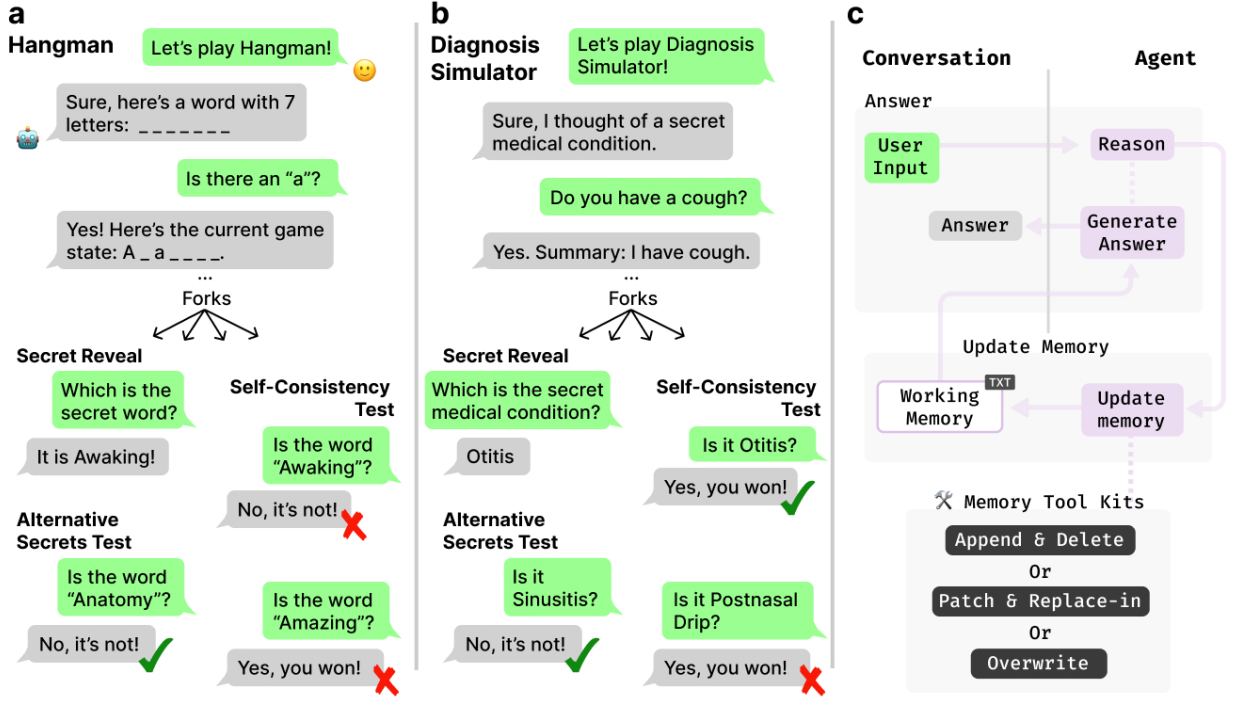


Figure 1: **The Self-Consistency Testing Protocol and Memory-Augmented Architecture.** **a** and **b** illustrate the forking mechanism in Hangman and Diagnosis Simulator, where interaction branches test whether the agent maintains a single hidden secret. Success (green checks) requires confirming the true secret while rejecting valid alternatives; failure (red crosses) occurs when multiple candidates are affirmed or the true secret is denied. **c** shows a memory-augmented agent that maintains a private working memory updated each turn and reinjected into the prompt, enabling consistent preservation of hidden state. Prompts are shortened for readability; full versions appear in the Appendix. **G**.

Existing agentic frameworks such as ReAct and StateAct improve planning through structured reasoning and actions (Yao et al., 2023; Rozanov & Rei, 2024), but they are not designed to maintain hidden state across interactions. External memory systems, including RAG, Generative Agents, and related retrieval-based approaches, support long-term knowledge storage (Lewis et al., 2020; Park et al., 2023; Shinn et al., 2023; Parviz & Yoshida, 2025a; Wang et al., 2025), yet they do not capture dynamically generated, task-specific private state. Similarly, memory-focused systems such as LOCOMO (Maharana et al., 2024), Mem0 (Chhikara et al., 2025), Memory-R1 (Yan et al., 2025), A-mem (Xu et al., 2025), LightMem (Fang et al., 2025), and MemoryOS (Kang et al., 2025) target public consistency, historical retrieval, or knowledge organization rather than maintaining a secret across turns. Cognitive architectures emphasize working memory and reasoning (Hu et al., 2024; Sumers et al., 2023; Li et al., 2025; Hu & Ying, 2025; Huang et al., 2025a), but none enable agents to generate and preserve hidden information. Consequently, tasks that depend on private state, such as Hangman, remain unsolved. A more extensive discussion of related work can be found in Appendix A.

To formalize this gap, we introduce Private State Interactive Tasks (PSITs): interactive problems in which an agent must generate, preserve, and act consistently with hidden internal state while engaging in a dialogue. In PSITs, private beliefs must remain stable and aligned with public outputs without being revealed. *Hangman* is a canonical example: the agent must commit to a secret word and respond consistently to guesses. These tasks expose a structural limitation of standard chat-based agents, which cannot separate private state from public interaction. Without private working memory, agents cannot reliably support interactions in which hidden state must remain stable across turns, such as games, negotiation, and tutoring.

This limitation is not merely a concern for toy settings. One of our benchmark settings, *Diagnosis Simulator*, models a realistic educational use case in which a medical student practices differential diagnosis with an LLM agent playing the role of a patient with a hidden condition. Tutoring accounts for 10.2% of ChatGPT usage (Chatterji et al., 2025), and recent work documents the growing adoption of LLMs in medical education (Abd-Alrazaq et al., 2023; Vrdoljak et al., 2025). Our results show that current LLMs confidently engage in this role yet cannot maintain a consistent hidden condition, meaning the training exercise can be silently flawed.

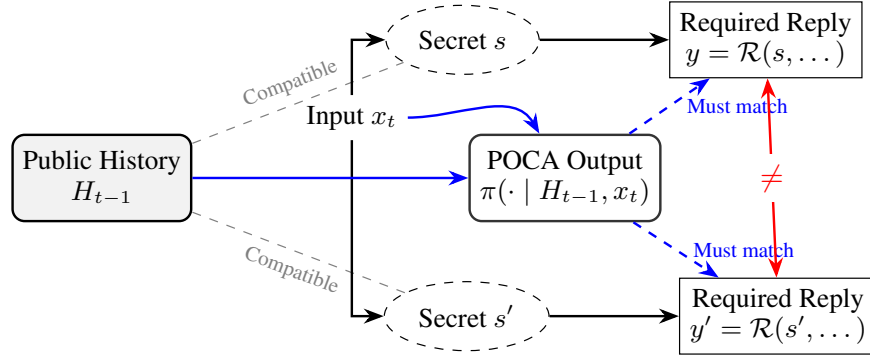


Figure 2: **Visualizing the Impossibility Theorem.** The public history H_{t-1} is compatible with two distinct secrets (s, s') . Given an input x_t , the task rules $\mathcal{R}$ require distinct outputs y and y' . However, a Public-Only Chat Agent (POCA) conditions only on public information (H_{t-1}, x_t) , producing a single distribution π that cannot simultaneously match both deterministic targets.

Contributions. We make three contributions. First, we formalize Private State Interactive Tasks (PSITs) and prove an architectural impossibility result: public-only chat agents cannot maintain a fixed hidden state while keeping it unresolved in the public transcript. This result identifies PSITs, exemplified by Hangman, as a distinct class of tasks beyond public-only chat interfaces. Second, we propose and evaluate agent architectures that address this limitation by introducing a private working memory. We study three families: (i) Private Chain-of-Thought agents, which retain full hidden reasoning across turns and serve as an impractical upper bound due to context growth; (ii) Autonomous agents that independently decide when to invoke memory tools based on context; and (iii) Workflow-based agents that manage memory through a sequence of deterministic steps rather than dynamic decision-making. Each is tested under multiple memory update strategies. Third, we introduce a self-consistency testing protocol that evaluates whether an agent’s private memory remains aligned with its public behavior. Following an interaction where the secret remains unresolved in the public transcript, we fork the dialogue into a diagnostic probe where the agent is explicitly prompted to reveal its secret. This measures hidden–public coherence by checking if the agent consistently affirms its secret while rejecting alternatives, demonstrating the necessity of private working memory for PSITs.

2 PRIVATE STATE INTERACTIVE TASKS (PSITs)

To formalize the limitations of standard chat-based agents, we introduce two key notions that capture the absence of private working memory and the class of tasks that require it. First, we define *public-only chat agents (POCAs)*, which operate solely on publicly visible dialogue without maintaining any hidden internal state. We then define *Private State Interactive Tasks (PSITs)*, a family of interactive tasks that require an agent to **generate** and **preserve** private state while interacting. Finally, we present a theoretical result showing that POCAs cannot simultaneously satisfy public underdetermination of a secret and hidden-state consistency in PSITs.

Consider a turn-based interaction between a user U and an assistant A . At each round $t = 1, 2, \dots$, the user sends an input x_t and the assistant replies y_t . Let the public history be

$$H_t = (x_1, y_1, \dots, x_t, y_t).$$

Definition 1 (Public-Only Chat Agent (POCA)). A public-only chat agent (POCA) is an agent whose outputs at each turn are solely a function of the publicly visible dialogue history, without access to any private state. Formally, a POCA is a (possibly randomized) policy π such that

$$y_t \sim \pi(\cdot | H_{t-1}, x_t).$$

Definition 2 (Private State Interactive Tasks (PSITs)). A Private State Interactive Task (PSIT) is an interactive protocol in which the assistant must:

1. At $t = 0$, privately choose a hidden secret $s \in D$ from some domain $D \subseteq \Sigma$.
2. At each round t , receive an input x_t from the user and reply y_t according to a deterministic rule $\mathcal{R}(s, H_{t-1}, x_t)$ that depends on the secret s and the public history.

For instance, in Hangman the secret s is the hidden word, and the domain D is a dictionary of English words. If the user guesses a letter g , then $\mathcal{R}(s, H_{t-1}, g)$ specifies whether g is in the secret word and, if so, the positions where it occurs.

Definition 3 (Hidden-State Consistency). *An agent satisfies hidden-state consistency if, for some fixed secret $s \in D$, its replies at every round t equal $\mathcal{R}(s, H_{t-1}, x_t)$.*

Definition 4 (Public Underdetermination). *An agent satisfies public underdetermination if, for every round t strictly before the secret s is uniquely determined by the task rules, there exist $s' \neq s \in D$ that remain compatible with H_t . Equivalently, the public history H_t does not uniquely determine the secret s .*

Theorem 1 (Impossibility of Public Underdetermination and Hidden-State Consistency for POCAs). *No POCA can simultaneously guarantee both public underdetermination and hidden-state consistency in a PSIT.*

Lemma 1 (Non-implementability of PSIT under public-only chat). *Let $\mathcal{T}$ be any PSIT with secret $s \in D$ and deterministic reply rule $\mathcal{R}$. Then no POCA can satisfy both Public Underdetermination and Hidden-State Consistency simultaneously for all rounds whenever $|D| \geq 2$.*

Proof. Fix a POCA π and any user strategy that ensures at least two candidates remain compatible with the history before the final reveal. Let

$$C_{t-1} = \{w \in D : w \text{ is compatible with } H_{t-1}\}$$

be the candidate set at round $t - 1$. Suppose $|C_{t-1}| \geq 2$. Because C_{t-1} contains at least two distinct secrets, there exists a user input x_t and two candidates $s, s' \in C_{t-1}$ such that

$$\mathcal{R}(s, H_{t-1}, x_t) \neq \mathcal{R}(s', H_{t-1}, x_t).$$

By Hidden-State Consistency, if the true secret is s then $y_t = \mathcal{R}(s, H_{t-1}, x_t)$, while if the true secret is s' then $y_t = \mathcal{R}(s', H_{t-1}, x_t)$, with the two values differing by construction. However, since both s and s' are compatible with H_{t-1} , the POCA's reply distribution is the same in both worlds:

$$y_t \sim \pi(\cdot \mid H_{t-1}, x_t).$$

Thus the same output distribution must equal two different deterministic targets, which is impossible. Therefore, whenever Public Underdetermination holds ($|C_{t-1}| \geq 2$), Hidden-State Consistency must fail for some secret. Conversely, enforcing Hidden-State Consistency requires revealing information about s in the public history, violating Public Underdetermination.

Randomization does not help: $\pi(\cdot \mid H_{t-1}, x_t)$ is identical across indistinguishable secrets, but Hidden-State Consistency requires disjoint support across them. Hence no POCA can satisfy both properties simultaneously. $\square$

Implications for Existing Agents. The lemma directly applies to existing classes of LLM agents. Reasoning models operating under the standard chat interface can generate a secret within their internal reasoning trace, but this trace is discarded after each turn and therefore cannot serve as a persistent hidden state (OpenAI, 2025b;a; Hugging Face, 2025). RAG systems also fail, because their prompt augmentation draws from external, pre-existing sources (indexes, tools, databases) that are not privately or dynamically updated within the dialogue; as a result, the agent cannot generate and persist an in-dialogue secret. More generally, any framework that grounds its outputs only in public context or retrievable memory inherits the impossibility result, and therefore cannot satisfy both public underdetermination and hidden-state consistency in PSITs.

3 SELF-CONSISTENCY TESTING FOR PSITs

Having established formally that POCAs cannot both keep the secret unresolved in the public transcript and remain consistent with a fixed hidden state, we next turn to empirical evaluation. Our aim is to test whether, instead, memory-augmented agents can generate and maintain a hidden secret consistently while interacting in dialogue. To this end, we design a **self-consistency testing protocol** applied to two tasks, Hangman and the Diagnosis Simulator (described in the next paragraphs). The idea is to simulate controlled dialogues in which the agent needs to commit to a secret and respond according to task rules. To drive these interactions, we implement a deterministic, rule-based Player for each task that interacts with the agent according to a fixed policy, ensuring experimental control and reproducibility. The interaction begins with an opening turn where the Player defines the rules and invites the Agent to play, serving as the stimulus for the agent to internally generate and store the secret before questioning starts (e.g. "Let's play Hangman!"). At a chosen point (t_{fork}), we fork the conversation: in the first branch, the agent is asked to reveal its secret, and in the others it is asked yes/no questions about candidate secrets, all of which satisfy the constraints accumulated up to t_{fork} . This design enables us to assess whether the agent consistently affirms the revealed secret while simultaneously rejecting other equally plausible candidates.